

UNIVERSITY ACADEMIC MANAGEMENT SYSTEM

Complete MySQL Database Code

```
/* -----
1. CREATE DATABASE
----- */

DROP DATABASE IF EXISTS University_Academic_Management;

CREATE DATABASE University_Academic_Management;

USE University_Academic_Management;

/* -----
2. DEPARTMENT MODULE
----- */

CREATE TABLE Department (
    Department_ID INT PRIMARY KEY,
    Department_Name VARCHAR(100) NOT NULL UNIQUE,
    HOD_Name VARCHAR(100)
);

/* -----
3. STUDENT MODULE
----- */

CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(100) NOT NULL,
    Gender VARCHAR(10),
    DOB DATE,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Department_ID INT NOT NULL,
    Year INT NOT NULL,
    Semester INT NOT NULL,

    FOREIGN KEY (Department_ID)
        REFERENCES Department(Department_ID)
);

/* -----
4. FACULTY MODULE
----- */

CREATE TABLE Faculty (
    Faculty_ID INT PRIMARY KEY,
    Faculty_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Designation VARCHAR(50),
    Department_ID INT NOT NULL,

    FOREIGN KEY (Department_ID)
```

```

REFERENCES Department(Department_ID)
);

/* -----
5. COURSE MODULE
----- */

CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(100) NOT NULL,
    Credits INT NOT NULL,
    Department_ID INT NOT NULL,
    Faculty_ID INT,

    FOREIGN KEY (Department_ID)
        REFERENCES Department(Department_ID),

    FOREIGN KEY (Faculty_ID)
        REFERENCES Faculty(Faculty_ID)
);

/* -----
6. ENROLLMENT MODULE
----- */

CREATE TABLE Enrollment (
    Enrollment_ID INT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Course_ID INT NOT NULL,
    Academic_Year VARCHAR(20) NOT NULL,
    Semester INT NOT NULL,

    FOREIGN KEY (Student_ID)
        REFERENCES Student(Student_ID),

    FOREIGN KEY (Course_ID)
        REFERENCES Course(Course_ID),

    UNIQUE(Student_ID, Course_ID, Academic_Year, Semester)
);

/* -----
7. ATTENDANCE MODULE
----- */

CREATE TABLE Attendance (
    Attendance_ID INT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Course_ID INT NOT NULL,
    Total_Classes INT NOT NULL,
    Attended_Classes INT NOT NULL,

```

```

        FOREIGN KEY (Student_ID)
            REFERENCES Student(Student_ID),

        FOREIGN KEY (Course_ID)
            REFERENCES Course(Course_ID)
    );

/* -----
8. EXAM MODULE
----- */

CREATE TABLE Exam (
    Exam_ID INT PRIMARY KEY,
    Exam_Name VARCHAR(50) NOT NULL,
    Academic_Year VARCHAR(20) NOT NULL,
    Semester INT NOT NULL,
    Exam_Date DATE
);

/* -----
9. MARKS / RESULT MODULE
----- */

CREATE TABLE Marks (
    Mark_ID INT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Course_ID INT NOT NULL,
    Exam_ID INT NOT NULL,
    Internal_Marks INT NOT NULL,
    External_Marks INT NOT NULL,
    Total_Marks INT NOT NULL,
    Grade VARCHAR(5),

    FOREIGN KEY (Student_ID)
        REFERENCES Student(Student_ID),

    FOREIGN KEY (Course_ID)
        REFERENCES Course(Course_ID),

    FOREIGN KEY (Exam_ID)
        REFERENCES Exam(Exam_ID),

    UNIQUE(Student_ID, Course_ID, Exam_ID)
);

/* -----
10. FEES MODULE
----- */

CREATE TABLE Fees (
    Fee_ID INT PRIMARY KEY,
    Student_ID INT NOT NULL,

```

```

        Academic_Year VARCHAR(20) NOT NULL,
        Semester INT NOT NULL,
        Total_Fees DECIMAL(10,2) NOT NULL,
        Paid_Fees DECIMAL(10,2) NOT NULL,
        Balance_Fees DECIMAL(10,2) NOT NULL,
        Payment_Status VARCHAR(20),

        FOREIGN KEY (Student_ID)
            REFERENCES Student(Student_ID)
    );

/* =====
   INSERT DEPARTMENT DATA
   ===== */

INSERT INTO Department
(Department_ID, Department_Name, HOD_Name)
VALUES
(1, 'Artificial Intelligence and Machine Learning', 'Dr. Ravi'),
(2, 'Computer Science Engineering', 'Dr. Kumar'),
(3, 'Electronics and Communication Engineering', 'Dr. Suresh'),
(4, 'Mechanical Engineering', 'Dr. Prasad');

/* =====
   INSERT STUDENT DATA
   ===== */

INSERT INTO Student
(Student_ID, Student_Name, Gender, DOB, Email, Phone,
 Department_ID, Year, Semester)
VALUES
(101, 'Sai Krishna', 'Male', '2005-06-15',
 'sai@gmail.com', '9876543210', 1, 2, 3),

(102, 'Rahul Kumar', 'Male', '2005-08-20',
 'rahul@gmail.com', '9876543211', 2, 2, 3),

(103, 'Anjali', 'Female', '2006-01-10',
 'anjali@gmail.com', '9876543212', 1, 2, 3),

(104, 'Priya', 'Female', '2005-11-05',
 'priya@gmail.com', '9876543213', 3, 2, 3),

(105, 'Arjun', 'Male', '2005-04-25',
 'arjun@gmail.com', '9876543214', 2, 2, 3);

/* =====
   INSERT FACULTY DATA
   ===== */

INSERT INTO Faculty
(Faculty_ID, Faculty_Name, Email, Phone,

```

```

Designation, Department_ID)
VALUES
(201, 'Dr. Arun', 'arun@university.com',
'9000000001', 'Professor', 1),

(202, 'Dr. Priya Sharma', 'priya.faculty@university.com',
'9000000002', 'Associate Professor', 2),

(203, 'Dr. Suresh', 'suresh@university.com',
'9000000003', 'Professor', 3),

(204, 'Dr. Lakshmi', 'lakshmi@university.com',
'9000000004', 'Assistant Professor', 1);

/* =====
   INSERT COURSE DATA
   ===== */

INSERT INTO Course
(Course_ID, Course_Name, Credits, Department_ID, Faculty_ID)
VALUES
(301, 'Database Management Systems', 4, 1, 201),
(302, 'Data Structures', 4, 1, 204),
(303, 'Object Oriented Programming', 3, 2, 202),
(304, 'Computer Networks', 3, 2, 202),
(305, 'Digital Electronics', 4, 3, 203);

/* =====
   INSERT ENROLLMENT DATA
   ===== */

INSERT INTO Enrollment
(Enrollment_ID, Student_ID, Course_ID, Academic_Year, Semester)
VALUES
(401, 101, 301, '2026-27', 3),
(402, 101, 302, '2026-27', 3),
(403, 102, 303, '2026-27', 3),
(404, 102, 304, '2026-27', 3),
(405, 103, 301, '2026-27', 3),
(406, 103, 302, '2026-27', 3),
(407, 104, 305, '2026-27', 3),
(408, 105, 303, '2026-27', 3);

/* =====
   INSERT ATTENDANCE DATA
   ===== */

INSERT INTO Attendance
(Attendance_ID, Student_ID, Course_ID,
Total_Classes, Attended_Classes)
VALUES
(601, 101, 301, 50, 45),

```

```

(602, 101, 302, 50, 42),
(603, 102, 303, 50, 40),
(604, 102, 304, 50, 38),
(605, 103, 301, 50, 43),
(606, 103, 302, 50, 46),
(607, 104, 305, 50, 41),
(608, 105, 303, 50, 44);

/* =====
   INSERT EXAM DATA
   ===== */

INSERT INTO Exam
(Exam_ID, Exam_Name, Academic_Year, Semester, Exam_Date)
VALUES
(701, 'Mid-I', '2026-27', 3, '2026-07-15'),
(702, 'Mid-II', '2026-27', 3, '2026-09-10'),
(703, 'End Semester', '2026-27', 3, '2026-11-20');

/* =====
   INSERT MARKS DATA
   ===== */

INSERT INTO Marks
(Mark_ID, Student_ID, Course_ID, Exam_ID,
 Internal_Marks, External_Marks, Total_Marks, Grade)
VALUES
(801, 101, 301, 701, 25, 65, 90, 'A+'),
(802, 101, 302, 701, 23, 60, 83, 'A'),
(803, 102, 303, 701, 22, 58, 80, 'A'),
(804, 102, 304, 701, 20, 55, 75, 'B+'),
(805, 103, 301, 701, 20, 55, 75, 'B+'),
(806, 103, 302, 701, 24, 62, 86, 'A'),
(807, 104, 305, 701, 21, 57, 78, 'B+'),
(808, 105, 303, 701, 23, 64, 87, 'A');

/* =====
   INSERT FEES DATA
   ===== */

INSERT INTO Fees
(Fee_ID, Student_ID, Academic_Year, Semester,
 Total_Fees, Paid_Fees, Balance_Fees, Payment_Status)
VALUES
(901, 101, '2026-27', 3, 100000, 100000, 0, 'Paid'),
(902, 102, '2026-27', 3, 100000, 75000, 25000, 'Partial'),
(903, 103, '2026-27', 3, 100000, 100000, 0, 'Paid'),
(904, 104, '2026-27', 3, 95000, 70000, 25000, 'Partial'),
(905, 105, '2026-27', 3, 100000, 50000, 50000, 'Partial');

/* =====

```

```

BASIC DISPLAY QUERIES
===== */

-- Display all departments
SELECT * FROM Department;

-- Display all students
SELECT * FROM Student;

-- Display all faculty
SELECT * FROM Faculty;

-- Display all courses
SELECT * FROM Course;

-- Display all enrollments
SELECT * FROM Enrollment;

-- Display all attendance records
SELECT * FROM Attendance;

-- Display all exams
SELECT * FROM Exam;

-- Display all marks
SELECT * FROM Marks;

-- Display all fee records
SELECT * FROM Fees;

/* =====
   STUDENT + DEPARTMENT DETAILS
===== */

SELECT
    S.Student_ID,
    S.Student_Name,
    S.Gender,
    S.Email,
    S.Phone,
    D.Department_Name,
    S.Year,
    S.Semester
FROM Student S
JOIN Department D
ON S.Department_ID = D.Department_ID;

/* =====
   COURSE + FACULTY DETAILS
===== */

SELECT
    C.Course_ID,

```

```

        C.Course_Name,
        C.Credits,
        D.Department_Name,
        F.Faculty_Name
FROM Course C
JOIN Department D
ON C.Department_ID = D.Department_ID
LEFT JOIN Faculty F
ON C.Faculty_ID = F.Faculty_ID;

/* =====
STUDENT ENROLLMENT DETAILS
===== */

SELECT
    S.Student_Name,
    C.Course_Name,
    E.Academic_Year,
    E.Semester
FROM Enrollment E
JOIN Student S
ON E.Student_ID = S.Student_ID
JOIN Course C
ON E.Course_ID = C.Course_ID;

/* =====
ATTENDANCE REPORT
===== */

SELECT
    S.Student_Name,
    C.Course_Name,
    A.Total_Classes,
    A.Attended_Classes,
    ROUND(
        (A.Attended_Classes / A.Total_Classes) * 100,
        2
    ) AS Attendance_Percentage
FROM Attendance A
JOIN Student S
ON A.Student_ID = S.Student_ID
JOIN Course C
ON A.Course_ID = C.Course_ID;

/* =====
STUDENTS WITH ATTENDANCE ABOVE 75%
===== */

SELECT
    S.Student_ID,
    S.Student_Name,
    C.Course_Name,

```

```

ROUND(
    (A.Attended_Classes / A.Total_Classes) * 100,
    2
) AS Attendance_Percentage
FROM Attendance A
JOIN Student S
ON A.Student_ID = S.Student_ID
JOIN Course C
ON A.Course_ID = C.Course_ID
WHERE
    (A.Attended_Classes / A.Total_Classes) * 100 >= 75;

/* =====
MARKS / RESULT REPORT
===== */

SELECT
    S.Student_ID,
    S.Student_Name,
    C.Course_Name,
    E.Exam_Name,
    M.Internal_Marks,
    M.External_Marks,
    M.Total_Marks,
    M.Grade
FROM Marks M
JOIN Student S
ON M.Student_ID = S.Student_ID
JOIN Course C
ON M.Course_ID = C.Course_ID
JOIN Exam E
ON M.Exam_ID = E.Exam_ID;

/* =====
STUDENTS SCORING MORE THAN 80
===== */

SELECT
    S.Student_Name,
    C.Course_Name,
    M.Total_Marks,
    M.Grade
FROM Marks M
JOIN Student S
ON M.Student_ID = S.Student_ID
JOIN Course C
ON M.Course_ID = C.Course_ID
WHERE M.Total_Marks > 80;

/* =====
FEE REPORT
===== */

```

```

SELECT
    S.Student_ID,
    S.Student_Name,
    F.Total_Fees,
    F.Paid_Fees,
    F.Balance_Fees,
    F.Payment_Status
FROM Fees F
JOIN Student S
ON F.Student_ID = S.Student_ID;

```

```

/* =====
STUDENTS WITH PENDING FEES
===== */

```

```

SELECT
    S.Student_ID,
    S.Student_Name,
    F.Balance_Fees,
    F.Payment_Status
FROM Fees F
JOIN Student S
ON F.Student_ID = S.Student_ID
WHERE F.Balance_Fees > 0;

```

```

/* =====
COUNT STUDENTS IN EACH DEPARTMENT
===== */

```

```

SELECT
    D.Department_Name,
    COUNT(S.Student_ID) AS Number_Of_Students
FROM Department D
LEFT JOIN Student S
ON D.Department_ID = S.Department_ID
GROUP BY
    D.Department_ID,
    D.Department_Name;

```

```

/* =====
COUNT COURSES IN EACH DEPARTMENT
===== */

```

```

SELECT
    D.Department_Name,
    COUNT(C.Course_ID) AS Number_Of_Courses
FROM Department D
LEFT JOIN Course C
ON D.Department_ID = C.Department_ID
GROUP BY
    D.Department_ID,

```

```

D.Department_Name;

/* =====
UPDATE EXAMPLES
===== */

-- Update student phone number
UPDATE Student
SET Phone = '9999999999'
WHERE Student_ID = 101;

-- Update student email
UPDATE Student
SET Email = 'saikrishna@university.com'
WHERE Student_ID = 101;

/* =====
VIEW 1 - STUDENT ACADEMIC REPORT
===== */

CREATE VIEW Student_Academic_Report AS
SELECT
    S.Student_ID,
    S.Student_Name,
    D.Department_Name,
    C.Course_Name,
    M.Total_Marks,
    M.Grade
FROM Student S
JOIN Department D
ON S.Department_ID = D.Department_ID
JOIN Marks M
ON S.Student_ID = M.Student_ID
JOIN Course C
ON M.Course_ID = C.Course_ID;

/* Display View */

SELECT * FROM Student_Academic_Report;

/* =====
VIEW 2 - ATTENDANCE REPORT
===== */

CREATE VIEW Student_Attendance_Report AS
SELECT
    S.Student_ID,
    S.Student_Name,
    C.Course_Name,
    A.Total_Classes,
    A.Attended_Classes,

```

```

        ROUND(
            (A.Attended_Classes / A.Total_Classes) * 100,
            2
        ) AS Attendance_Percentage
FROM Attendance A
JOIN Student S
ON A.Student_ID = S.Student_ID
JOIN Course C
ON A.Course_ID = C.Course_ID;

/* Display View */

SELECT * FROM Student_Attendance_Report;

/* =====
   STORED PROCEDURE 1
   SEARCH STUDENT
   ===== */

DELIMITER //

CREATE PROCEDURE SearchStudent(IN SID INT)
BEGIN
    SELECT
        S.Student_ID,
        S.Student_Name,
        S.Email,
        S.Phone,
        D.Department_Name,
        S.Year,
        S.Semester
    FROM Student S
    JOIN Department D
    ON S.Department_ID = D.Department_ID
    WHERE S.Student_ID = SID;
END //

DELIMITER ;

/* Run Procedure */

CALL SearchStudent(101);

/* =====
   STORED PROCEDURE 2
   SHOW STUDENT MARKS
   ===== */

DELIMITER //

CREATE PROCEDURE StudentMarks(IN SID INT)

```

```

BEGIN
    SELECT
        S.Student_Name,
        C.Course_Name,
        M.Internal_Marks,
        M.External_Marks,
        M.Total_Marks,
        M.Grade
    FROM Marks M
    JOIN Student S
    ON M.Student_ID = S.Student_ID
    JOIN Course C
    ON M.Course_ID = C.Course_ID
    WHERE S.Student_ID = SID;
END //

DELIMITER ;

/* Run Procedure */

CALL StudentMarks(101);

/* =====
   FINAL DATABASE CHECK
   ===== */

SHOW TABLES;

SELECT 'University Academic Management System Created Successfully'
AS Project_Status;

```